Algoritmo parallelo di Smith-Waterman per allineamento di sequenze DNA

Christian Pozzoli 29678A

Ottobre 2025



Indice

1 Introduzione **3**
1.1 Allineamento di sequenze di DNA 3
1.2 Obiettivo del progetto 3
1.3 Strumenti e ambiente di sviluppo 4

2 Algoritmo **4**
2.1 Algoritmo di Smith-Waterman 4
2.2 Implementazione sequenziale 4

3 Implementazione su GPU **4**
3.1 Parallelizzazione dell'algoritmo 4
3.2 Implementazione su GPU 4
3.3 Ottimizzazione con blocchi 4
3.4 Ottimizzazione con memoria condivisa (SMEM) 4
3.5 Ottimizzazione con warp e coalescenza 4
3.6 Analisi delle prestazioni e comparazione 4

4 Conclusione **4**

1 Introduzione

1.1 Allineamento di sequenze di DNA

L'allineamento di sequenze di DNA è una delle operazioni fondamentali della bioinformatica e rappresenta uno strumento essenziale per lo studio comparativo dei genomi. L'obiettivo principale dell'allineamento è confrontare due o più sequenze di basi di nucleotidi al fine di individuare delle regioni di somiglianza che possono riflettere relazioni evolutive, funzionali o strutturali tra gli organismi.

In termini pratici, un allineamento consiste nel disporre le sequenze una sopra l'altra, introducendo eventuali spazi vuoti ("gap") per massimizzare il numero di corrispondenze tra le basi. Questa operazione consente di identificare mutazioni, inserzioni o delezioni, e di dedurre l'origine comune di geni o proteine. Queste analisi sono fondamentali in vari ambiti applicativi, tra cui

- l'annotazione di nuovi genomi
- la ricerca di omologie tra geni
- la previsione della funzione di proteine sconosciute
- lo studio delle malattie genetiche

Esistono due principali categorie di allineamento: globale e locale. L'allineamento globale, proposto da Needleman e Wunsch (1970), confronta le sequenze nella loro interezza, ed è più adatto quando le sequenze hanno lunghezze simili e un alto grado di somiglianza. L'allineamento locale, introdotto da Smith e Waterman (1981), mira invece a trovare le regioni più simili all'interno di sequenze potenzialmente molto diverse, risultando particolarmente utile in contesti biologici reali, dove solo alcune porzioni delle sequenze mostrano omologia.

1.2 Obiettivo del progetto

L'obiettivo del progetto è implementare e ottimizzare l'algoritmo di Smith-Waterman sfruttando la potenza di calcolo parallelo delle GPU attraverso la piattaforma CUDA. L'algoritmo di Smith-Waterman presenta una complessità computazionale di tipo quadratico rispetto alla lunghezza delle sequenze da confrontare. Questo lo rende poco adatto all'elaborazione di grandi quantità di dati, come quelli generati dalle moderne tecniche di sequenziamento del DNA.

L'obiettivo principale è quindi ridurre i tempi di esecuzione attraverso una parallelizzazione efficiente delle sue operazioni e mantenendo inalterata la precisione dell'algoritmo.

Il progetto mira a dimostrare come l'uso del calcolo parallelo possa rendere praticabile l'utilizzo dell'algoritmo di Smith-Waterman anche in contesti reali e su dataset di grandi dimensioni.

1.3 Strumenti e ambiente di sviluppo

Lo sviluppo del progetto è stato condotto utilizzando il linguaggio CUDA C, sfruttando l'integrazione diretta con l'architettura di calcolo parallelo delle GPU NVIDIA.

L'ambiente di sviluppo è stato configurato su sistema operativo Windows, utilizzando il compilatore `nvcc` in versione 12.9.86, incluso nel CUDA Toolkit 13.0. Il sistema era dotato dei driver NVIDIA in versione 581.57, pienamente compatibili con la versione del toolkit utilizzato.

Per la fase di test e analisi delle prestazioni sono state impiegate due GPU di architettura recente:

- NVIDIA GeForce RTX 3070 (Compute Capability 8.6)
- NVIDIA GeForce RTX 4060 (Compute Capability 8.9)

2 Algoritmo

L'algoritmo di Smith-Waterman permette di ottenere un allineamento locale ottimale tramite un approccio di programmazione dinamica. In questa sezione viene descritto il funzionamento dell'algoritmo, la sua formulazione e la relativa implementazione sequenziale, che farà da riferimento per la successiva versione parallela su GPU.

2.1 Algoritmo di Smith-Waterman

2.2 Implementazione sequenziale

3 Implementazione su GPU

3.1 Parallelizzazione dell'algoritmo

3.2 Implementazione su GPU

3.3 Ottimizzazione con blocchi

3.4 Ottimizzazione con memoria condivisa (SMEM)

3.5 Ottimizzazione con warp e coalescenza

3.6 Analisi delle prestazioni e comparazione

4 Conclusione